

SOLUTION ARCHITECTURE PROJET ONSSA

Tu peux faire :

```
Serveur ONSSA
→ pipeline documentaire
→ corpus
→ embeddings
→ Qdrant
→ retriever
→ top chunks pertinents
→ envoyer seulement ces chunks au Cloud GPU
→ Qwen3 génère la réponse
```

Donc au lieu d'envoyer tout le corpus au cloud, tu envoies seulement **la petite partie utile après retrieval**.

Pourquoi c'est possible ?

Parce que dans un RAG, le LLM n'a pas besoin de voir tous les documents.

Il a seulement besoin des **passages pertinents** récupérés par le retriever.

Exemple :

```
Question :
"Quelles sont les obligations sanitaires selon la loi 28-07 ?"
```

```
Serveur ONSSA :
retrouve les 5 passages les plus pertinents
```

```
Cloud GPU :
reçoit seulement ces 5 passages + la question
génère la réponse avec citations
```

Donc le cloud GPU ne reçoit pas :

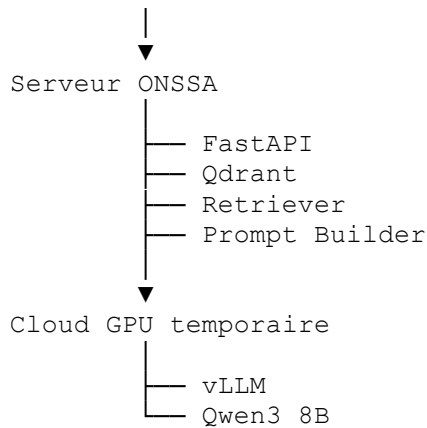
```
tous les PDF
tout le corpus
toute la base Qdrant
```

Il reçoit seulement :

```
question utilisateur
+ top 5 chunks
+ métadonnées/citations
```

Architecture que tu proposes

Plateforme ONSSA



Le serveur ONSSA fait :

document processing
corpus
embeddings
Qdrant
retrieval
prompt construction
citations

Le cloud GPU fait seulement :

LLM inference
réponse finale

C'est quoi l'avantage ?

Très important :

1. Moins de données envoyées au cloud

Au lieu d'envoyer 1 000 documents, tu envoies seulement 5 à 10 passages par question.

2. Le corpus reste chez ONSSA

La base documentaire complète reste locale.

Corpus complet : ONSSA
Qdrant : ONSSA
Cloud : seulement contexte temporaire

3. Test plus proche de la future architecture

Parce que plus tard, quand ONSSA aura un GPU local, tu remplaces seulement :

Cloud GPU

par :

GPU local ONSSA

Le reste ne change pas.

4. Moins de stockage cloud

Le cloud GPU n'a presque rien à stocker. Il héberge juste :

- Qwen3 ;
- vLLM ;
- éventuellement tokenizer/cache ;
- logs temporaires.

Pourquoi je ne l'ai pas recommandé en premier ?

Parce que cette option est **meilleure architecturalement**, mais un peu plus compliquée techniquement.

Elle demande que :

Serveur ONSSA puisse appeler l'API du Cloud GPU

Donc il faut :

- accès internet sortant depuis le serveur ONSSA ;
- autorisation firewall ;
- endpoint cloud sécurisé ;
- API key ;
- HTTPS ;
- éventuellement VPN/tunnel ;
- gestion de la latence ;
- protection des logs.

Pour un test simple, copier le corpus dans le cloud est plus facile.

Mais pour une validation plus propre avec ONSSA, **ton idée est meilleure** si le réseau le permet.

Les deux architectures possibles

Option A — Tout tester sur cloud GPU

Cloud GPU

- corpus
- embeddings
- Qdrant
- retriever
- vLLM

└─ Qwen3

Avantage :

- simple ;
- rapide ;
- pas besoin de connecter ONSSA au cloud.

Inconvénient :

- tu copies le corpus complet ;
- moins proche d'une architecture souveraine.

Option B — Retrieval local + LLM cloud

Serveur ONSSA

```
├─ corpus
├─ embeddings
├─ Qdrant
├─ retriever
└─ prompt builder
```



Cloud GPU

```
├─ vLLM
└─ Qwen3
```

Avantage :

- plus propre ;
- corpus reste local ;
- cloud fait seulement le calcul LLM ;
- architecture proche de la production future ;
- moins de stockage cloud.

Inconvénient :

- nécessite connexion réseau sortante ;
- configuration API cloud ;
- latence réseau ;
- sécurité endpoint.

Exemple concret d'un appel

Le serveur ONSSA construit ce payload :

```
{
  "question": "Quelles sont les obligations sanitaires selon la loi 28-07
?",
  "context": [
```

```
{
  "chunk_id": "loi_28_07_article_3_chunk_1",
  "text": "Texte réglementaire pertinent...",
  "source": "Loi 28-07",
  "article": "Article 3"
},
{
  "chunk_id": "loi_28_07_article_4_chunk_1",
  "text": "Autre passage pertinent...",
  "source": "Loi 28-07",
  "article": "Article 4"
}
],
"instructions": "Réponds uniquement à partir du contexte fourni et cite les sources."
}
```

Le cloud GPU retourne :

```
{
  "answer": "Selon la Loi 28-07, les obligations sanitaires incluent...",
  "usage": {
    "latency_seconds": 3.2
  }
}
```

Puis le serveur ONSSA ajoute/contrôle les citations et renvoie à la plateforme.

Dans ce cas, qu'est-ce qui reste sur le cloud ?

Seulement :

```
vLLM
Qwen3 8B
API d'inférence
logs temporaires
```

Pas besoin de mettre :

```
PDF
corpus complet
Qdrant complet
base documentaire complète
```

Est-ce que les embeddings doivent être sur le cloud ?

Non.

Dans cette architecture, les embeddings restent locaux avec Qdrant.

```
Embeddings + Qdrant = serveur ONSSA
LLM + GPU = cloud temporaire
```

Le cloud n'a pas besoin des embeddings. Il reçoit déjà les textes finaux.

Donc quelle architecture recommander ?

Pour demain, je te conseille de présenter **deux niveaux** :

Niveau 1 — Test rapide

Tout sur Cloud GPU avec corpus public

Pour aller vite, 1 jour à 3 jours.

Niveau 2 — Test plus propre

Retrieval local ONSSA + LLM cloud temporaire

Pour valider une architecture proche production.

Et tu dis :

Ma préférence technique est la deuxième option si le serveur ONSSA peut faire des appels sortants sécurisés vers le cloud GPU. Dans ce cas, le corpus et Qdrant restent chez ONSSA, et le cloud GPU sert uniquement à générer la réponse avec le modèle de langage.

Phrase simple pour le responsable

Oui, on peut éviter d'envoyer tout le corpus vers le cloud. Le serveur ONSSA peut faire tout le traitement documentaire, construire le corpus, créer les embeddings, héberger Qdrant et récupérer les passages pertinents. Ensuite, il envoie seulement la question et les 5 ou 10 passages utiles au GPU cloud, qui exécute le modèle Qwen3 et retourne la réponse. C'est plus propre, car le cloud est utilisé uniquement comme moteur de calcul temporaire.

Ma recommandation finale

Si ONSSA accepte que le serveur appelle internet :

Option recommandée :

Serveur ONSSA = pipeline + corpus + Qdrant + retriever + API

Cloud GPU = vLLM + Qwen3 uniquement

Si ONSSA ne veut pas ouvrir de connexion réseau :

Option simple :

Exporter corpus public vers Cloud GPU et tester tout là-bas

Ton idée est donc correcte : **on peut envoyer seulement les informations extraites après retrieval**, et c'est même l'architecture hybride la plus propre.